

Revisão de arquitetura para o GLPI Data Service

Este relatório revisa a proposta de arquitetura apresentada anteriormente, incorporando o diagrama da estrutura atual e apontando ajustes para atender melhor aos requisitos de multi-contexto e multi-projeto. O objetivo permanece organizar o backend em camadas claras, reduzir duplicação de código e facilitar a adição de novos contextos (DTIC, SIS, Saude...) e novos módulos (tickets, carregadores, equipamentos...).

1. Situação atual e problemas

O repositório atual foi visualizado através de um diagrama ASCII que ilustra a pilha de camadas e evidencia diversos pontos frágeis. O serviço GLPI Data Service utiliza FastAPI e múltiplos bancos PostgreSQL (schemas `dtic` e `sis`). A estrutura de pastas contém uma camada **API** subdividida em **core** (rotas compartilhadas), **dtic**, **sis** e um conjunto de rotas *legacy* que aceitam o parâmetro `{context}`. A seguir um resumo dos problemas mais evidentes, baseados no diagrama fornecido (ver arquivo `current_architecture_diagram.md`):

1. **Duplicação massiva** – o arquivo `routes/search.py` possui cerca de 575 linhas com implementações duplicadas para DTIC e SIS. Cada par de funções (`search_tickets` / `search_sis_tickets`, `suggest_values` / `suggest_sis_values`, etc.) difere apenas pelo contexto usado na consulta ao banco. Isso aumenta manutenção e risco de divergência de comportamentos.
2. **Inconsistência de padrões** – há três formas diferentes de parametrizar o contexto: (i) arquivos como `dtic/dashboard.py` fixam o contexto (“dtic”) de forma hardcoded; (ii) alguns endpoints utilizam o path parameter `{context}`; (iii) alguns módulos duplicam rotas para cada contexto (`/dtic/search` e `/sis/search`). A ausência de um padrão único dificulta a implementação de novos módulos e gera confusão.
3. **Services layer incompleta** – apenas `dashboard_service.py` está presente na camada de serviços. Funções de negócio para tickets, busca, sincronização e auditoria estão diretamente nas rotas, violando o princípio de separação de preocupações e dificultando testes.
4. **Ausência de padrão para novos projetos** – não está claro onde criar arquivos para novos domínios como “equipamentos” (parte do SIS) ou domínios compartilhados. Existem múltiplas opções de localização e nenhuma regra consolidada.

Esses problemas resultam em código redundante, difícil de manter e de escalar. A seguir apresentamos uma proposta revisada que amplia a discussão anterior.

2. Principais requisitos de multi-tenancy e contexto

Conforme discutido anteriormente, multi-tenancy significa executar uma aplicação para vários clientes ou contextos, isolando dados e experiência ¹. O projeto atual assemelha-se ao modelo de bancos separados (DTIC e SIS), mas pode migrar para um modelo de multi-schema no mesmo banco para

reduzir duplicação e gestão de infraestrutura ². Independente do modelo, alguns princípios se aplicam:

- **Identificação do contexto em cada requisição:** por subdomínio, cabeçalho HTTP (X-Context), token JWT ou parte do path. A escolha deve ser única e obrigatória para todas as rotas ³.
- **Isolamento de dados:** cada contexto opera em seu schema ou banco; transações não se misturam; queries sempre filtram pelo tenant.
- **Escalabilidade:** adicionar contextos não pode exigir clonagem de código. A nova camada ou schema deve ser configurada pela infraestrutura, enquanto as camadas de domínio e aplicação permanecem as mesmas.
- **Auditoria e autorização:** mecanismos de auditoria devem registrar qual contexto executou cada operação e quem é o usuário envolvido ⁴.

3. Revisão da arquitetura proposta

A estrutura sugerida anteriormente baseada em **Clean Architecture** e **Ports & Adapters** continua válida; aqui refinamos alguns pontos e mapeamos como ela resolve os problemas identificados. A estrutura global recomendada é:

```
src/
├── core/                                # Utilidades e serviços compartilhados
│   ├── config.py                       # Configuração e carregamento de .env
│   ├── logging.py
│   ├── glpi_client/
│   │   ├── client.py                  # Cliente HTTP para GLPI comum a todos os contextos
│   │   └── ...
│   └── db/
│       ├── base.py                   # SQLAlchemy base comum
│       ├── session.py                # Fábrica de sessões multi-contexto
│       └── schemas.py                 # Criação dinâmica de schemas
├── contexts/
│   ├── dtic/
│   │   ├── domain/                  # Entidades e objetos de valor
│   │   │   └── ticket.py
│   │   ├── application/
│   │   │   ├── services/
│   │   │   │   └── ticket_service.py
│   │   │   └── use_cases/
│   │   │       └── submit_ticket.py
│   │   ├── infrastructure/
│   │   │   ├── repositories/
│   │   │   │   └── ticket_repository_impl.py
│   │   │   └── adapters/
│   │   │       └── postgres_repository.py
│   │   └── interface/
│   │       ├── api/
│   │       │   ├── router.py          # Inclui rotas do contexto DTIC
│   │       │   └── projects/
```

```

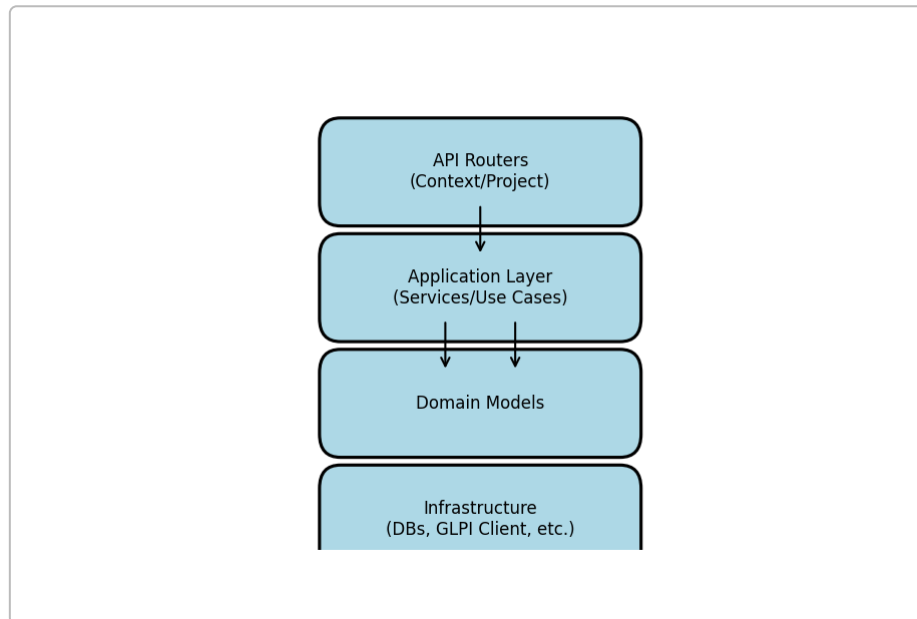
├── tickets.py
├── schemas.py          # Pydantic models
├── sis/
│   ├── domain/
│   │   └── ticket.py
│   ├── application/
│   │   ├── services/
│   │   │   └── ticket_service.py
│   │   ├── use_cases/
│   │   │   └── create_carregador.py
│   ├── infrastructure/
│   │   └── repositories/
│   │       └── ticket_repository_impl.py
│   └── interface/
│       ├── api/
│       │   ├── router.py          # Rotas SIS
│       │   └── projects/
│       │       ├── tickets.py
│       │       └── carregadores.py
│       └── schemas.py
├── main.py              # Cria a aplicação e registra routers de cada contexto
└── tests/

```

Melhorias em relação à estrutura atual:

- **Camada core mais rica** – move módulos como `postgres_manager.py` e `glpi_client/client.py` para `core/db/` e `core/glpi_client/`, permitindo reutilização e evitando duplicação de conexões. Nessa camada também se definem funções utilitárias para criar/metadados para cada schema (multi-schema) conforme a abordagem descrita no guia do app-generator ⁵.
- **Separação por contextos** – a pasta `contexts/` agrupa código de cada contexto. Isso isola regras específicas (por exemplo, regras do SIS que não se aplicam ao DTIC) e facilita adicionar novos contextos como `saude` ou `educacao`. Cada contexto inclui suas camadas de **domínio**, **application**, **infrastructure** e **interface**, seguindo DDD ⁶.
- **Serviços e casos de uso explícitos** – a camada de `application/services` substitui a lógica de negócio atualmente espalhada em rotas (`search.py`, `tickets.py`, etc.). Cada serviço recebe dependências (repositórios, clientes externos) via injeção e implementa métodos reutilizáveis. Os **use cases** (como `submit_ticket.py`) orquestram sequências complexas.
- **Repositórios como ports** – no domínio, definem-se interfaces de repositório (`TicketRepository`) que especificam as operações necessárias. A infraestrutura fornece implementações (adapters) para PostgreSQL ou outra fonte. Isso concretiza o padrão Ports & Adapters ⁷.
- **Router por contexto** – substitui o padrão atual de múltiplos arquivos em `api/dtic` e `api/sis` e elimina as rotas *legacy* duplicadas. Cada router importa os mesmos serviços, recebendo o contexto via dependência. Assim, os endpoints para tickets, busca, sync e audit são implementados uma única vez e expostos com prefixos diferentes (`/api/v1/dtic/...`, `/api/v1/sis/...`). Quando necessário, pode-se manter a forma `/api/v1/{context}/...` e validar o parâmetro contra uma enum com valores permitidos.

O diagrama abaixo ilustra essa organização e destaca o fluxo entre camadas. A camada de interface se comunica apenas com `application/services`, que usam repositórios do domínio e adaptadores de infraestrutura. A camada de domínio é isolada de detalhes externos.



4. Estratégias para eliminar duplicação

4.1 Identificação única do contexto

Escolha um único mecanismo para identificar o contexto em todas as rotas. As opções são:

- **Cabeçalho HTTP** (`X-Context` ou `X-Tenant`): o cliente envia `dtic` ou `sis` e o backend seleciona o schema. É limpo e não polui o path.
- **Parâmetro de rota** (`/api/v1/{context}/tickets`): o contexto é parte da URL. Simples de entender, mas resulta em URIs mais longas. A aplicação deve validar que `{context}` pertence à enum de contextos válidos.
- **Subdomínio** (`dtic.api.example.com`): cada contexto acessa a mesma aplicação via subdomínio. Pode ser interessante para produção, mas complica testes locais.

Uma vez escolhido, crie uma dependência FastAPI para resolver o contexto. Exemplo para o modelo de cabeçalho:

```
from enum import Enum
from fastapi import Header, HTTPException, Depends
from typing import Generator
from sqlalchemy.orm import Session

class ContextEnum(str, Enum):
    dtic = "dtic"
    sis = "sis"

async def get_context(x_context: str = Header(...)) -> ContextEnum:
    try:
```

```

        return ContextEnum(x_context)
    except ValueError:
        raise HTTPException(status_code=400, detail="Invalid context")

async def get_db_session(context: ContextEnum) -> Generator[Session, None, None]:
    # Escolha a engine ou schema conforme o contexto
    if context == ContextEnum.dtic:
        session = SessionLocal_dtic()
    else:
        session = SessionLocal_sis()
    try:
        yield session
    finally:
        session.close()

```

Todos os serviços recebem `context` e `db_session` via `Depends`. Assim, uma função `search_tickets` de um `TicketService` pode receber o contexto e decidir qual repositório concreto usar. Não há necessidade de duplicar a função para cada contexto.

4.2 Reutilização de serviços entre contextos

Implementações de serviços (`TicketService`, `SearchService`, `SyncService`, `AuditService`) não devem conter código específico de DTIC ou SIS. Elas operam com interfaces de repositórios e clientes GLPI. Para cada contexto, injete a implementação de repositório que usa o schema correspondente. Por exemplo:

```

class TicketService:
    def __init__(self, repository: TicketRepository):
        self.repository = repository

    async def search(self, query: str) -> list[Ticket]:
        return await self.repository.search(query)

# No domínio
from typing import Protocol

class TicketRepository(Protocol):
    async def search(self, query: str) -> list[Ticket]: ...

# Na infraestrutura
class PostgresTicketRepository(TicketRepository):
    def __init__(self, session: Session, schema: str):
        self.session = session
        self.schema = schema
    async def search(self, query: str) -> list[Ticket]:
        # Constrói a consulta no schema adequado
        return (self.session
                .query(TicketModel)
                .filter_by(name=query)

```

```

        .execution_options(schema_translate_map={None: self.schema})
        .all())

@router.get("/tickets")
async def list_tickets(context: ContextEnum = Depends(get_context),
                       session: Session = Depends(get_db_session)):
    repo = PostgresTicketRepository(session, schema=context.value)
    service = TicketService(repo)
    tickets = await service.search("")
    return tickets

```

Essa abordagem elimina o par `search_tickets` / `search_sis_tickets` e similares. O mesmo serviço pode ser instanciado com diferentes repositórios dependendo do contexto.

4.3 Modularização de projetos dentro de um contexto

Para projetos específicos (por exemplo, o módulo **carregadores** no SIS), siga o mesmo padrão de camadas. Dentro de `contexts/sis/application/services/` crie `carregador_service.py` e em `interface/api/projects/` crie `carregadores.py`. Esses módulos podem compartilhar entidades com outros contextos se a lógica for igual, ou definir suas próprias entidades.

Ao adicionar um novo projeto **equipamentos** no SIS:

1. **Domain:** defina a entidade `Equipamento` e interfaces de repositório em `contexts/sis/domain/equipamento.py`.
2. **Application:** implemente `EquipamentoService` em `services/` e casos de uso em `use_cases/`.
3. **Infrastructure:** crie `equipamento_repository_impl.py` com consultas SQLAlchemy para o schema `sis`.
4. **Interface:** adicione `contexts/sis/interface/api/projects/equipamentos.py` com rotas REST.

Para projetos multi-contexto (ex.: relatórios que funcionam em DTIC e SIS), posicione as entidades em `core/` ou em uma pasta compartilhada (`contexts/shared`) e implemente repositórios que funcionem em qualquer schema.

5. Escalabilidade e futuras extensões

O desenho modular facilita a expansão:

- **Novo contexto "saude"** – crie `contexts/saude/` com suas camadas. Configure um novo schema ou banco "saude" e inclua o router em `main.py`. As dependências de contexto e sessão de banco lidarão com a seleção do schema.
- **Funcionalidades transversais** – funcionalidades como relatórios, auditoria ou sincronização podem ser implementadas em `core/` como serviços que recebem um contexto e operam em qualquer schema. Workers assíncronos (Celery, RQ) podem trabalhar independentemente e acessar os schemas corretos via dependências.
- **Adição de novos bancos** – embora recomendemos multi-schema para reduzir overhead, a arquitetura suporta múltiplos bancos: basta configurar `SessionLocal_saude()` e injetar o

repositório correspondente. Caso futuramente deseje migrar para multi-schema, a camada de infraestrutura é o único ponto a ser ajustado.

6. Exemplos adicionais

Para enriquecer a pesquisa, além dos repositórios já mencionados (App-Generator multi-tenancy, fastapi-todo-ddd e hexagonal-architecture-with-python), explorei o repositório interno `dados_agents_copilots` na organização informada. Nele existem módulos que integram GLPI (`CleanAuthProject/app/services/glpi.py`), com exemplos de serviços que separam o cliente HTTP e a lógica de autenticação. Esse padrão pode ser seguido: o cliente GLPI fica em `core/glpi_client`, enquanto serviços específicos são implementados em cada contexto. Não há, entretanto, exemplos de multi-contexto completo, reforçando a necessidade de implementar a estrutura descrita.

7. Prós e contras revisados

Aspecto	Vantagens	Desvantagens
Separação por camadas (DDD/Hex)	Facilita testes unitários, trocas de tecnologias, modificação isolada de camadas; suporta múltiplos contextos sem duplicação ⁸ .	Pode parecer complexa para quem não está acostumado; mais arquivos e interfaces ⁹ .
Contextos como módulos independentes	Isolamento forte; fácil adicionar contextos novos; reduz riscos de vazamento de lógica entre clientes.	Pode causar redundância se serviços compartilhados não forem centralizados; exige configuração de roteamento em <code>main.py</code> .
Multi-schema no mesmo banco	Compartilha hardware, simplifica backup e migrações; isola dados por schema; permite consultas cross-schema se necessário ² .	Gera complexidade nas migrações e possivelmente performance degradada se muitos schemas.
Rotas com único mecanismo de contexto	Permite reutilizar lógica sem duplicação; reduz código repetido; simplifica testes.	Requer validação rígida do contexto; exige alteração das rotas existentes, causando pequena quebra de API.
Serviços genéricos e repositórios	Lógica de negócio centralizada e reutilizável; fácil implementar mock em testes; adaptável a vários bancos.	Necessita de configuração adicional para injetar dependências corretas; pode parecer abstrato demais para iniciantes.

8. Conclusão revisada

A revisão confirma que a arquitetura em camadas proposta anteriormente continua adequada e reforça pontos práticos para resolver os problemas identificados no diagrama atual. Ao migrar para uma estrutura organizada por contextos e camadas, adotando um único mecanismo de identificação de contexto e repositórios genéricos, o projeto GLPI Data Service ganhará em escalabilidade, legibilidade e facilidade de manutenção. A eliminação de duplicação no arquivo `search.py` e em outras rotas se dará naturalmente ao centralizar a lógica em serviços e repositórios. A criação de novos contextos ou projetos exigirá apenas a adição de pastas e configuração de routers, sem alterar o core da aplicação.

1 3 4 Building Scalable Multi-Tenant Architectures in FastAPI | by Mahdi Jafari | Python in Plain English

<https://python.plainenglish.io/building-scalable-multi-tenant-architectures-in-fastapi-9b5457543e65?gi=10e65395212f>

2 5 Multitenancy with FastAPI - A practical guide — Documentation

<https://app-generator.dev/docs/technologies/fastapi/multitenancy.html>

6 Implementing Domain-Driven Design with FastAPI | by Hyunil Kim | 딜리버스 | Medium

<https://medium.com/delivus/implementing-domain-driven-design-with-fastapi-6aed788779af>

7 8 9 Hexagonal architecture in Python (FastAPI example included). | by Szymon Miks | Medium

<https://medium.com/@miks.szymon/hexagonal-architecture-in-python-e16a8646f000>